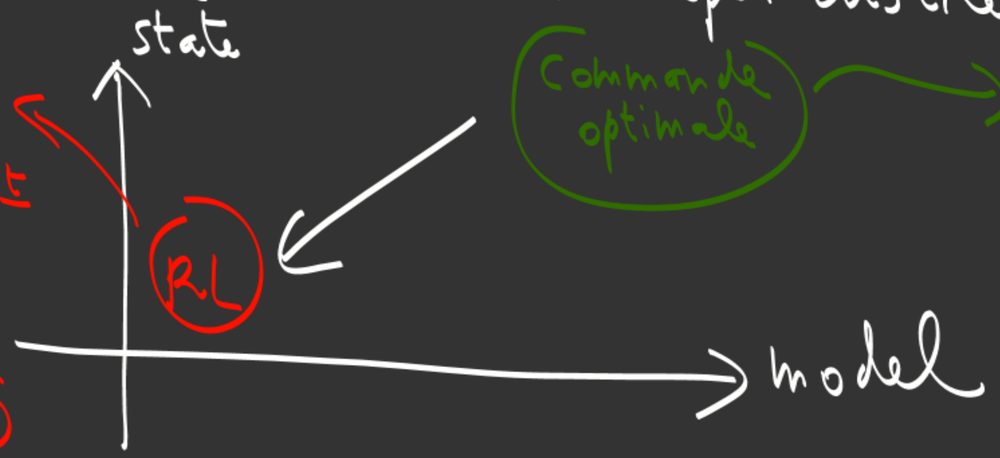
5. Reinforcement learning (RL)

Cadre général (learning) :

- apprentissage non-supervisé (clustering)
- —— " —— supervisé (SVM...)
- RL (→ commande opt. discrète sto. : Q-learning)

maps :
- carte enrichie itérativement
- état : position imprécise (GPS...)



Commande optimale → modèle $\dot{x}(t) = f(x(t), u(t))$ et état connus, $\int_0^{t_f} f^0(x(t), u(t))\,dt \to \min$

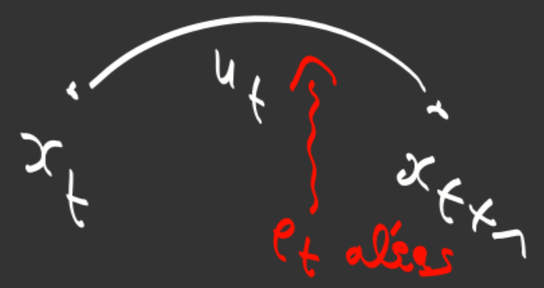
Approximate dynamic programming :

on suppose que notre état $x_t \in \mathbb{R}^n$, avec t temps discret ($t = 0, 1, \dots$), vérifie une dynamique stochastique

$$\begin{cases} x_{t+1} = f(x_t, u_t, e_t) \\ x_0 \text{ connu} \\ u_t \in U \subset \mathbb{R}^m \end{cases}$$

action / contrôle

aléas (processus à temps discret)



On a de plus une notion de récompense ("reward") modélisée, en horizon fini N ($= t_f$) comme suit :

$$E\left(\sum_{t=0}^{N} R(x_t, u_t)\right) \longrightarrow \max$$

f. aléas e_t dans dynamique

Commande optimale stochastique discrète

<u>Problème</u> : on ne connait pas (bien...) le modèle, ie on n'a en général pas accès à f, R...

→ Model based RL (on apprend f et R sur la base de réalisations $\tilde{x}_t, \tilde{u}_t$...)

→ ADP

ADP : - on ne cherche pas à apprendre le modèle...
- mais on se souvient que le problème sous-jacent est de nature comm. optim. discrète sto.
- on cherche à apprendre une approximation de sa fonction valeur

On se place en horizon infini ($N \longrightarrow \infty$) :

$$\underbrace{(1-\gamma)}_{\text{(normalisation)}} \cdot E\left(\sum_{t=0}^{\infty} \gamma^t \cdot R(x_t, u_t) \right) \longrightarrow \max \quad , \gamma \in]0,1[$$

"discount factor"

$$x_{t+1} = f(x_t, u_t, e_t)$$

x_0 connu , $u_t \in U$

MDP : Markov Decision Process

Déf. : (Q-function)

$$Q(x,u) := \max_{\substack{u_1,\dots \\ \in U}} \left\{ E\left(\sum_{t=0}^{\infty} \gamma^t \cdot R(x_t,u_t)\right) \;\middle|\; \begin{array}{l} x_{t+1} = f(x_t,u_t,e_t),\ t \geq 0 \\ x_0 = x,\ u_0 = u \end{array} \right\}$$

<u>Remarques</u> : i) la valeur $v(x) := \max_{u_0 \in U} Q(x,u_0)$

ii) l'intérêt d'apprendre Q est d'obtenir une politique ("policy") qui indique quelle action choisir dans l'état x_t :

$$u_t \leftarrow \overline{\pi}(x_0,\dots,x_t,u_0,\dots,u_{t-1}) \in \arg\max_{u \in U} Q(x_t,u)$$

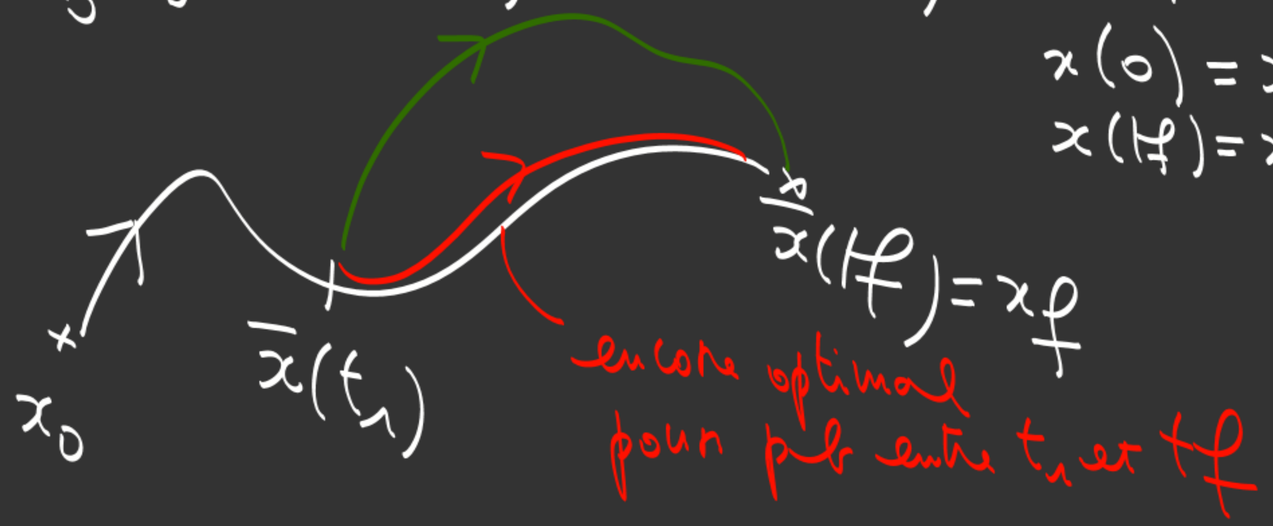
Pour apprendre cette fonction Q, on fait appel au principe de dynamique :

<u>Th.</u> (Bellman) : soit $\bar{x} : [0, t_f] \longrightarrow \mathbb{R}^n$ trajectoire opt. du pb

$$\int_0^{t_f} f^0(x(t), u(t))\, dt \longrightarrow \min, \quad \dot{x}(t) = f(x(t), u(t)), \; t \in [0, t_f]$$

$$x(0) = x_0, \quad u(t) \in U$$

$$x(t_f) = x_f$$



Alors, $\forall t_1 \in]0, t_f[$, la restriction $\bar{x}|_{[t_1, t_f]}$ reste optimale pour le pb sur $[t_1, t_f]$.

Ici, en discret, on a :

$$Q(x,u) = \max_{u_1 \in U, u_2 \in U, \dots} \left\{ E\left(\sum_{t=0}^{\infty} \gamma^t \cdot R(x_t, u_t)\right) \;\middle|\; \begin{array}{l} x_{t+1} = f(x_t, u_t, e_t), t \geq 0 \\ x_0 = x, u_0 = u \end{array} \right\}$$

$$= \underbrace{R(x,u)}_{t=0} + E\left(\underbrace{\max_{u_1, u_2 \dots \in U}}_{\max_{u_1 \in U} \max_{u_2 \in U}, \dots} \left\{ \underbrace{E\left(\sum_{t=1}^{\infty} \gamma^t \cdot R(x_t, u_t)\right)}_{\gamma \cdot \sum_{t=0}^{\infty} \gamma^t \cdot R(x_{t+1}, u_{t+1})} \;\middle|\; \begin{array}{l} x_{t+1} = f(x_t, u_t, e_t), t \geq 0 \\ x_1 = f(x, u, e_0) \end{array} \right\} \right)$$

$$\Rightarrow Q(x,u) = R(x,u) + \gamma E\Big(\max_{u' \in U} \max_{u_2 \in U, \dots} \Big\{ E\Big(\sum_{t=0}^{\infty} \gamma^t \cdot R(x_{t+1}, u_{t+1})\Big) \Big| \begin{array}{l} x_{t+1} = f(x_t, u_t, e_t), t \geq 1 \\ x_1 = f(x, u, e_0),\ u_1 = u' \end{array} \Big\}\Big)$$

$\Rightarrow Q$ vérifie l'équation de pt fixe :

$$Q(x,u) = R(x,u) + \gamma E\Big(\max_{u' \in U} Q(f(x,u,e_0), u')\Big)$$

$Q(f(x,u,e_0), u')$

R et f inconnus $\longrightarrow$ réalisation : trajectoire

"un épisode" $\{(\tilde{x}_0, \dots, \tilde{x}_L), (\tilde{u}_0, \dots, \tilde{u}_{L-1}), \overbrace{(\tilde{r}_0, \dots, \tilde{r}_{L-1})}^{\text{rewards}}$

Apprentissage d'un estimateur $\hat{Q}$:

$\hat{Q}(x,u,y)$: y paramètre ($\hat{Q}(x,u,y) = \underline{NN}(x,u,\underset{\text{poids réseau}}{y})$)

Neural Net

$y_k \longrightarrow y_{k+1}$ (amélioration estimateur) : "learning rate" $\longrightarrow$ "deep Q-learning"

$$\hat{Q}(\tilde{x}_t, \tilde{u}_t, y_{k+1}) := (1-\eta_k) \cdot \left[\tilde{r}_t + \gamma \max_{u' \in U} \hat{Q}(\tilde{x}_{t+1}, u', y_k) \right] + \eta_k \cdot \hat{Q}(\tilde{x}_t, \tilde{u}_t, y_k), \quad t = 0, \dots, L-1$$

on cherche y_{k+1}

Si $(\eta_k)_k \longrightarrow 0$, si $(y_k)_k \longrightarrow \bar{y}$, alors $\hat{Q}(\cdot,\cdot,\bar{y})$ vérifie l'équation de pt fixe voulue. Politique : $u_t \in \arg\max_{u \in U} \hat{Q}(x_t, u, y_k)$